

Custom Process SVG Renderer

Architecture handover for rebuilding the local deterministic process map renderer.

This document describes the local business process map renderer used by the AI Knowledge and Analytics Assistant. It is intended as a practical handover note for rebuilding a similar renderer in another project.

Purpose

The renderer turns extracted process knowledge into a business process diagram without using Lucid, Mermaid rendering, React Flow, BPMN.js, D3, or any external SaaS diagram tool.

It is a deterministic local SVG renderer. The backend calculates layout, nodes, edges, animation steps, and a static SVG. The frontend either injects the returned static SVG or redraws the same chart JSON as React SVG primitives for the animated walkthrough.

Source Files

Main renderer:

```
services/process_diagram/engine.py
```

FastAPI sidecar:

```
services/process_diagram/app.py
```

Process draft to diagram payload adapter:

```
src/assistant/process/diagram.py
```

Static frontend display:

```
frontend/src/ProcessDiagramPanel.tsx  
frontend/src/ProcessRegistryPage.tsx
```

Animated frontend redraw:

```
frontend/src/AnimatedProcessDiagramPanel.tsx
```

Mermaid metadata generator, not the live renderer:

```
src/assistant/process/maps.py
```

Runtime Shape

The renderer runs as a local FastAPI sidecar service:

```
.venv/bin/python -m uvicorn services.process_diagram.app:app --host 127.0.0.1 --port  
5300 --reload
```

Important endpoints:

```
GET /health  
GET /examples  
GET /examples/{example_id}/svg  
POST /process-chart/render  
POST /process-chart/render.svg
```

The main app calls the local sidecar through `ProcessDiagramClient`. Non-loopback hosts are refused, because the diagram service is intended to be local-only.

Input Contract

The service accepts either:

1. A structured `process_model`.
2. A plain narrative, which is converted into a simple model by deterministic heuristics.

The structured model is preferred.

Example request:

```
{
  "style": "plain",
  "format": "cross-functional-flowchart",
  "animation": true,
  "process_model": {
    "title": "Supplier setup",
    "nodes": [
      { "id": "buyer", "type": "lane", "label": "Category Buyer" },
      { "id": "support", "type": "lane", "label": "Trading Support" },
      { "id": "submit_form", "type": "task", "label": "Complete supplier setup form",
        "lane": "buyer" },
      { "id": "review", "type": "task", "label": "Review submitted form", "lane":
        "support" },
      { "id": "decision", "type": "gateway", "label": "Details complete?", "lane":
        "support" }
    ],
    "edges": [
      { "from": "submit_form", "to": "review", "label": "submit" },
      { "from": "review", "to": "decision", "label": "check" }
    ]
  }
}
```

Supported node types:

```
lane
start
end
task
gateway
who
system
control
risk
annotation
```

Input lane nodes represent owners or support categories. The renderer creates visual who nodes automatically for task owners.

Output Contract

The JSON endpoint returns a chart object containing:

```
chart_id
title
style
format
nodes[]
edges[]
animation_steps[]
narration_script[]
warnings[]
```

Each rendered node includes:

```
id
type
label
lane
x
y
width
height
metadata
```

Each rendered edge includes:

```
id
from
to
label
type
points[]
```

The SVG endpoint returns a complete <svg> document.

Layout Model

The layout is deterministic and coordinate-based. It uses fixed columns:

```
SYSTEM_X   = 86
PROCESS_X  = 470
WHO_X      = 850
```

Vertical flow starts at:

```
TOP_MARGIN = 88
ROW_GAP    = 190
```

Default sizes:

```
TASK_WIDTH    = 280
TASK_HEIGHT   = 112
EVENT_WIDTH   = 260
EVENT_HEIGHT  = 126
ROLE_WIDTH    = 265
ROLE_HEIGHT   = 92
SYSTEM_WIDTH  = 260
SYSTEM_HEIGHT = 86
GATEWAY_SIZE  = 66
```

Primary flow nodes are stacked vertically in the process column. Supporting nodes such as systems, controls, risks, and annotations are grouped beside the relevant primary node. Role or owner nodes are placed on the right.

The rough visual layout is:

systems / controls		process flow		who / owner
-----		-----		-----
system box		start		
system box	--->	task	---	owner box
		gateway		
control box	--->	task	---	owner box
		end		

Normalisation Rules

Before layout, the renderer:

1. Validates that every node has an id and readable label.

2. Normalises ids to lowercase snake-style identifiers.
3. Adds generated start and end nodes if missing.
4. Adds default lane nodes where needed.
5. Adds default sequential edges if none are supplied.
6. Maps source edge aliases from from / to into internal from_node / to_node fields.

The chart id is generated by hashing the normalised process model.

Shape Rendering

The SVG is assembled as a string in Python. It uses only native SVG elements.

Start and end:

```
polygon with angled sides
purple stroke
```

Task:

```
rounded rectangle
green stroke
centered text
```

Gateway:

```
circle
two crossing diagonal lines
```

Who and system:

```
tab-card rectangle
yellow stroke for who
blue stroke for system
inner vertical and horizontal lines
```

Control, risk, annotation:

```
rounded rectangle
dashed stroke
orange for control
red for risk
grey for annotation
```

Edges:

```
SVG path
right-angle routing where needed
arrow marker
optional white-backed label
```

Colours:

task stroke	#50c463
event stroke	#b126e8
gateway stroke	#374151
edge stroke	#334155
who stroke	#ffdd33
system stroke	#66adff
control stroke	#f59e0b
risk stroke	#ef4444
annotation stroke	#9ca3af
text	#111827

Text Handling

The renderer has its own simple text wrapping:

1. Estimate max characters per line from node width and font size.
2. Split labels by words.
3. Build lines until the max character count is reached.
4. Render each line as a separate SVG `<text>` element.
5. Increase node height when the wrapped text would exceed the minimum height.

Important constants:

```
TEXT_WIDTH_FACTOR = 0.56
line_height       = font_size + 6
```

The static SVG renderer escapes text with `html.escape` before inserting labels.

Edge Routing

Edges are calculated from node geometry:

1. If source and target are nearly vertically aligned, draw a direct vertical line from bottom to top.
2. If target is to the right, start at the source right edge and end at the target left edge.
3. If target is to the left, start at the source left edge and end at the target right edge.
4. Use a midpoint x coordinate to create right-angle connector paths.

The output edge stores its complete list of points so both backend SVG and frontend animation can use the same geometry.

Animation Model

If `animation` is enabled, the backend returns animation steps:

```
draw_node
draw_edge
```

Each step includes:

```
step
action
target_id
label
narration
```

The animated frontend does not use the backend SVG. Instead, it uses the returned chart JSON and redraws the nodes and edges as React SVG elements. It reveals flow nodes step by step, includes related owner/system/control/risk nodes, and scrolls the diagram frame to the active node.

Frontend Display

Static process maps are rendered by injecting the local SVG string:

```
<div dangerouslySetInnerHTML={{ __html: diagram.svg }} />
```

Animated maps are rendered as a normal React `<svg>` using the same chart JSON:

```
<svg viewBox={`0 0 ${dimensions.width} ${dimensions.height}`}>
  {chart.edges.map(...)}
  {chart.nodes.map(...)}
</svg>
```

Rebuild Checklist

To rebuild this renderer elsewhere:

1. Define a strict process model schema for nodes and edges.
2. Normalise ids, lanes, start/end nodes, and default edges before rendering.
3. Keep layout deterministic and data-driven.
4. Store final x/y/width/height on every node.
5. Store final route points on every edge.
6. Render static SVG from the same chart JSON.
7. Let the frontend render the returned SVG for static diagrams.
8. For animation, redraw the JSON as frontend SVG rather than trying to animate the raw SVG string.
9. Add text wrapping before drawing.
10. Grow node height based on wrapped text.
11. Keep all external text escaped before inserting into SVG.
12. Keep the service local-only unless a secure remote rendering contract is added.

Known Limitations

This renderer is intentionally simple. It does not currently provide:

```
automatic graph layout
swimlane bands
drag/drop editing
BPMN semantics
collision avoidance beyond the deterministic layout rules
zoom/pan controls beyond normal scrollable containers
advanced text measurement using browser or font metrics
```

If the next version needs more complex routing, lane layouts, or interactive editing, useful upgrade paths would be:

```
ELK.js for automatic graph layout
React Flow for interactive node editing
BPMN.js for BPMN-specific diagrams
Mermaid only for quick generated diagrams, not for precise product UI control
```

The current custom SVG approach is strongest when diagrams should be local, predictable, branded, simple to test, and independent from third-party diagram services.